APS Failure Classification: Tree-Based Methods

Haresh Sambandamoorthy | DSCI 552 | Homework 6

1. Tree-Based Methods

1(a) APS Failure dataset

The APS data contains 170 numeric readings per truck. The target is whether the air-pressure system has failed. The supplied training and test sets were kept separate.

Item	Training data	Test data
Rows	60,000	16,000
Positive (APS failure)	1,000	375
Negative	59,000	15,625
Missing feature entries	850,015 (8.33%)	228,680 (8.41%)
Number of features	170	170

1(b)(i) Handling missing values

Common approaches are deleting incomplete rows, filling blanks with a mean or median, or estimating missing values using nearby rows. Deleting rows would waste too much data here. I used the **median of each training feature** to fill blanks. The same saved training medians were used for test rows. For cross-validation, medians were recalculated from only the training portion of each fold.

1(b)(ii) Coefficient of variation

For each of the 170 features, CV = sample standard deviation / mean, using n - 1 in the standard deviation. All values are in **results/all_170_feature_cv.csv**. Values can be very large when a mean is close to zero. The 13 highest-CV features are:

cf_000, co_000, ad_000, cs_009, dh_000, dj_000, ag_000, as_000, ay_009, ak_000, az_009, ch_000, au_000.

The 13 features are used for plots only. All 170 features remain available for modeling.

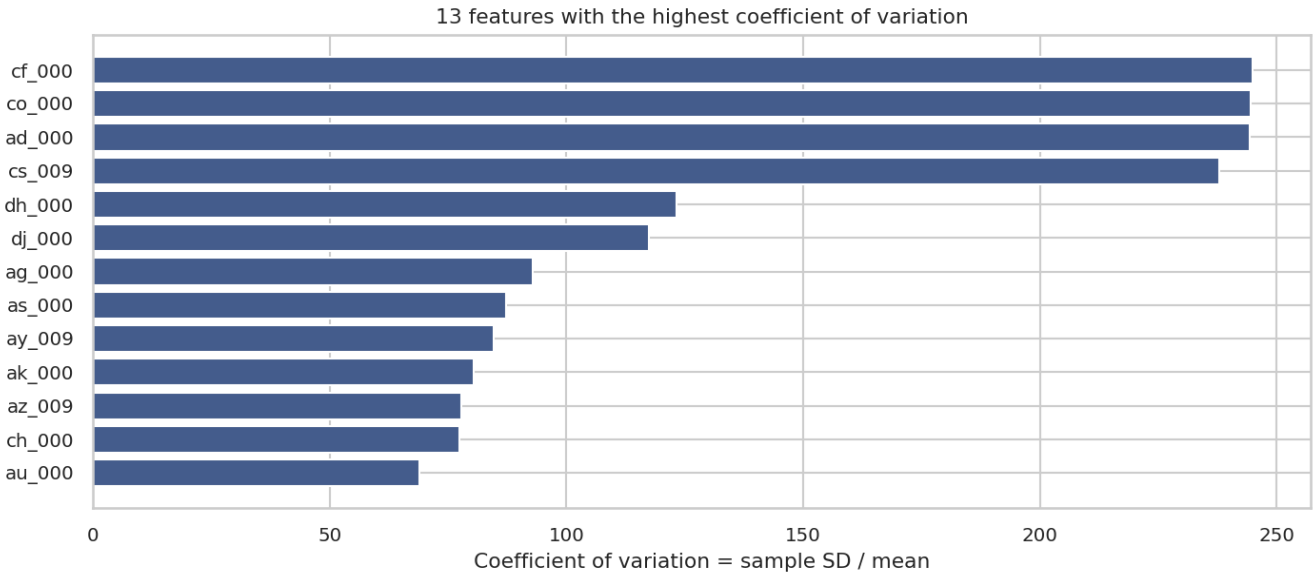


Figure 1. The thirteen highest-CV features; values are based on the training data.

1(b)(iii) Correlation matrix

The next chart shows correlations across all 170 features. A strong positive or negative correlation means two readings often change together; correlation alone does not prove that a reading is useful for predicting failures.

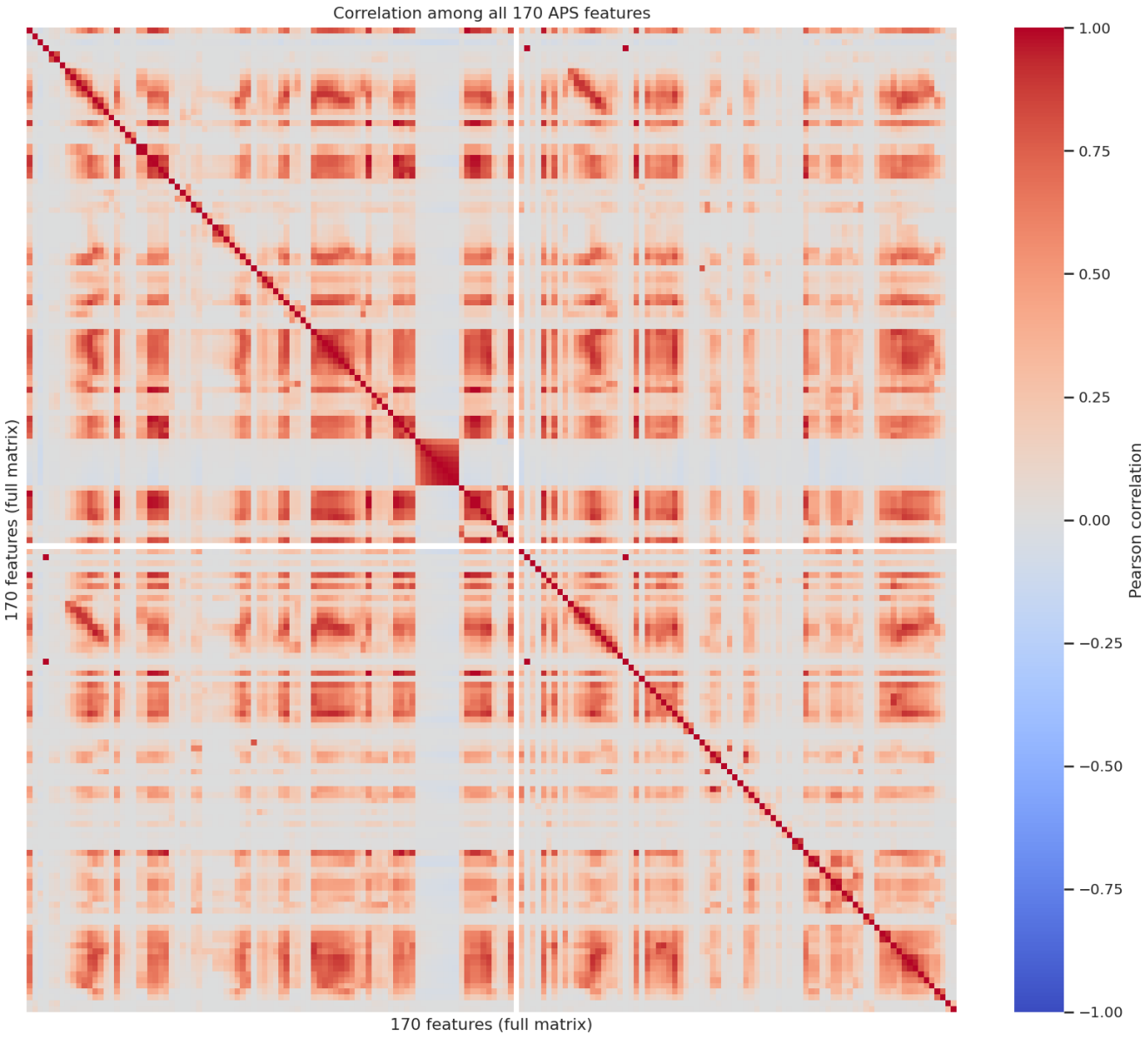


Figure 2. Correlation matrix for all 170 features.

1(b)(iv) Scatter plots and box plots

The assignment asks for floor(sqrt(170)) = 13 features. For readability, the scatter plots display a fixed sample of 250 positive and 250 negative observations. The box plots use all 60,000 training rows. A log-like plotting scale reduces the visual effect of extremely large values.

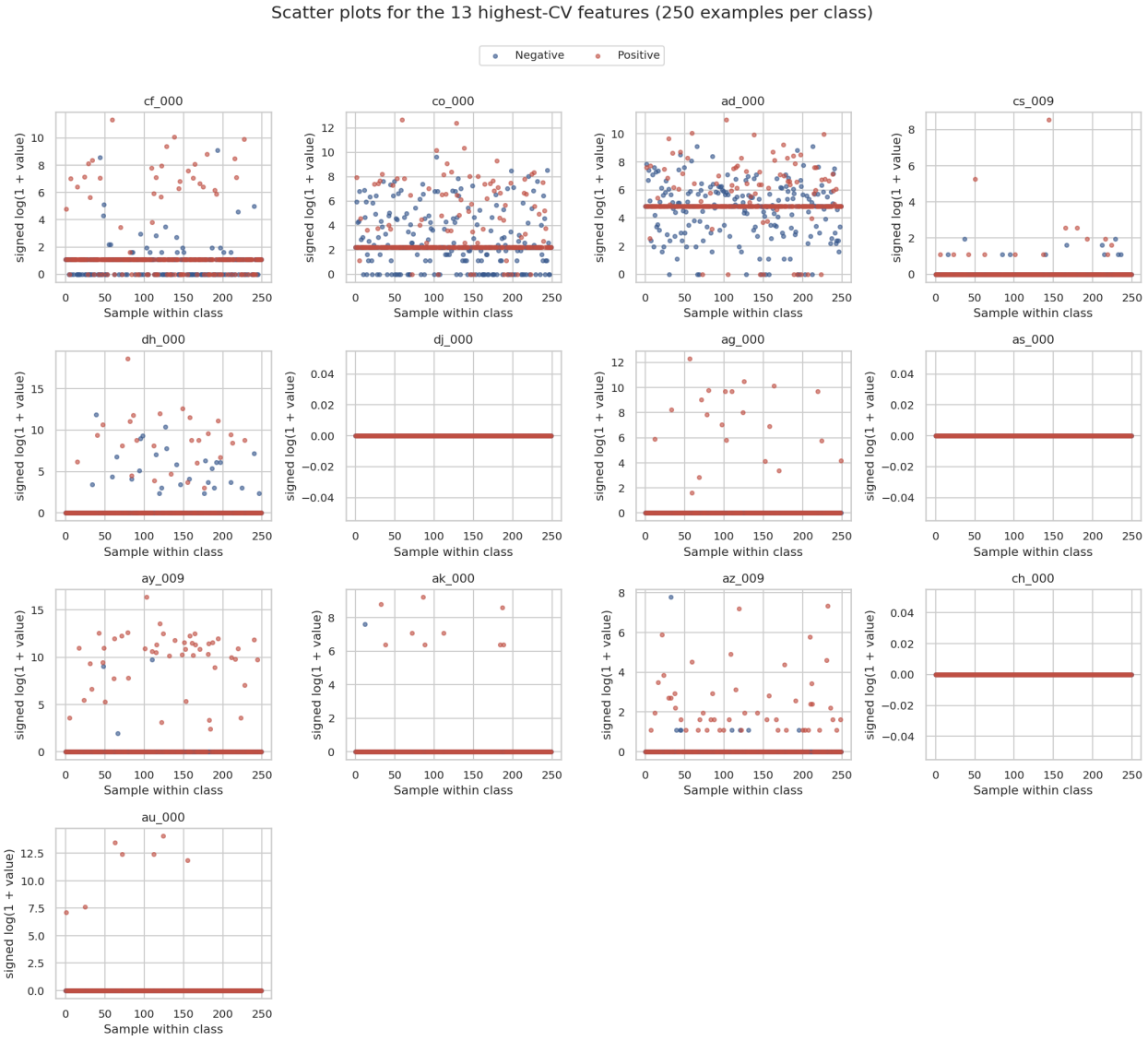


Figure 3. Scatter plots of the thirteen features with the largest CV.

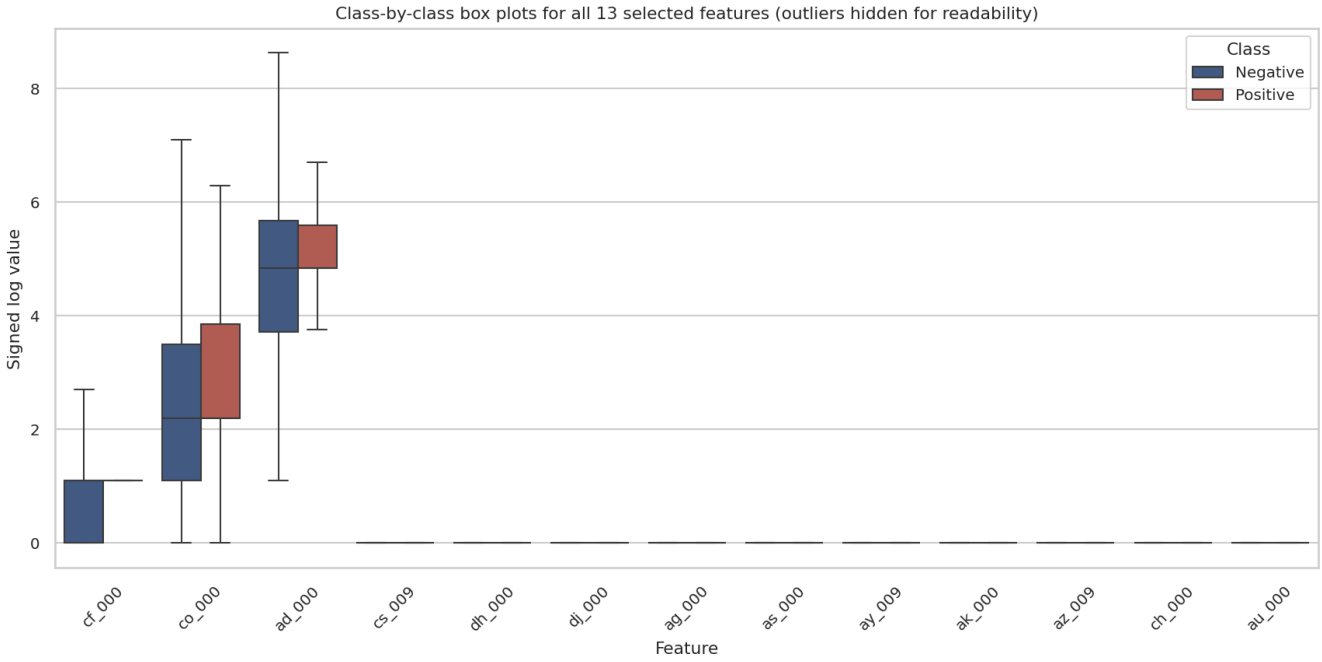


Figure 4. Box plots comparing positive and negative classes for all 13 features. Extreme points are hidden from this figure, not from the analysis.

Some plots suggest a difference between failure and non-failure observations, but the plots alone cannot tell us whether a feature is statistically important. High CV may also reflect a few unusually large numbers. We retain every feature in the models.

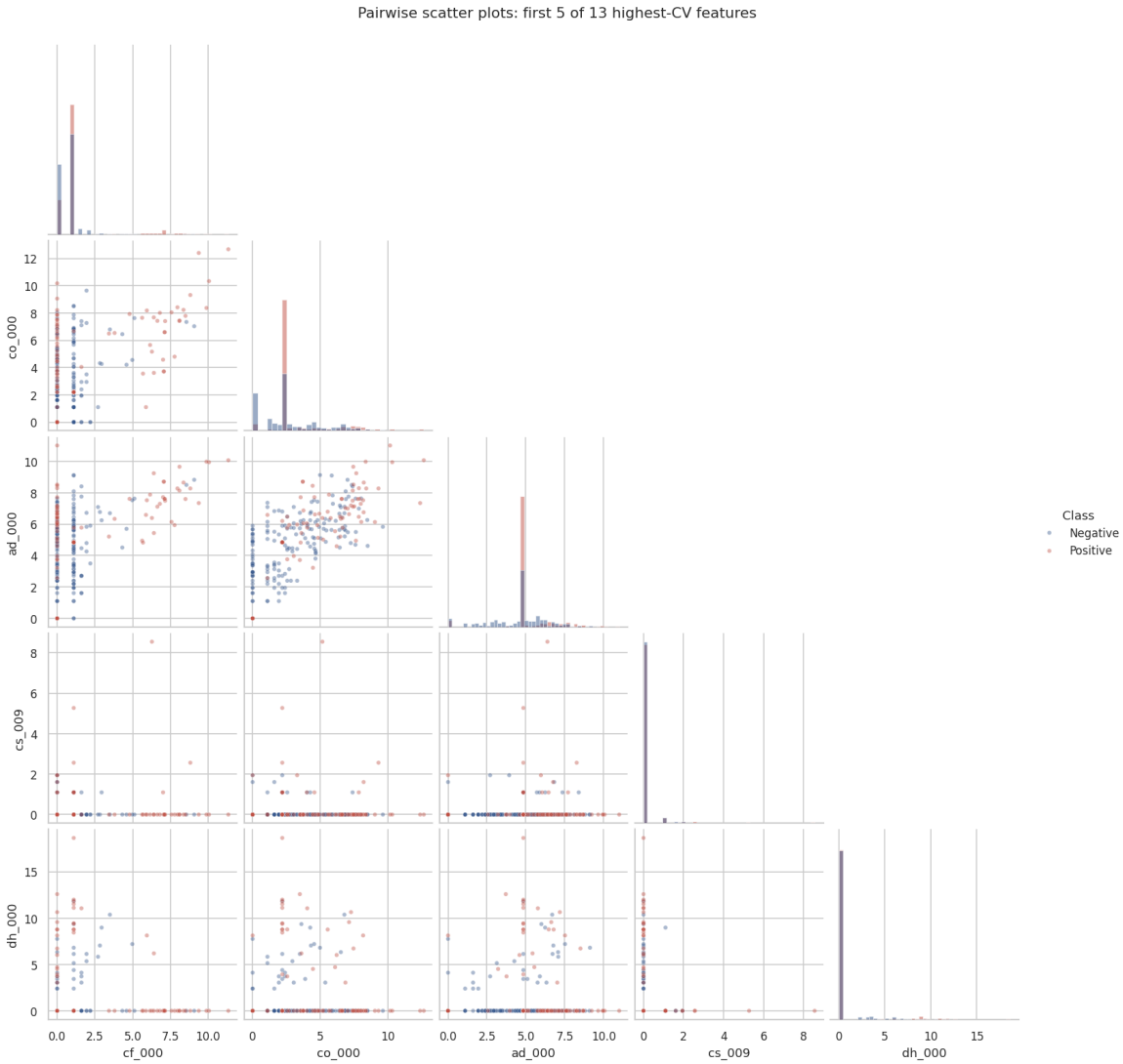


Figure 5. Pairwise scatter plots for five of the thirteen high-CV features.

1(b)(v) Class counts and imbalance

There are **1,000 positives (1.67%)** and **59,000 negatives (98.33%)** in the training data. That is 59 negatives for each positive. This dataset is highly imbalanced: a model predicting only the negative class would appear accurate but would fail to catch any APS failures.

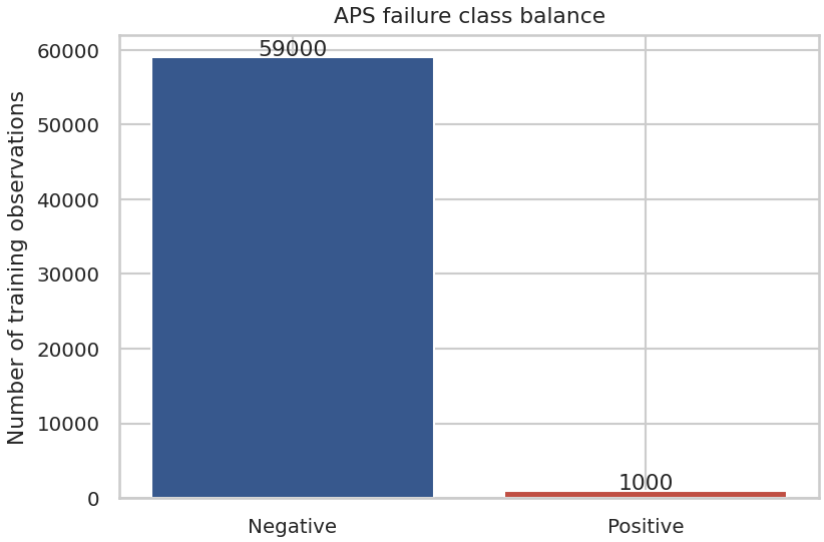


Figure 6. Training-set class counts.

Reading the model results

TN means a normal truck correctly called normal; FP means a false alarm; FN means a real failure missed; and TP means a correctly detected failure. Error is the share of wrong predictions. ROC-AUC shows how well the model ranks failures ahead of normal trucks. All confusion matrices use a fixed 0.50 threshold.

1(c) Random forest without compensation

The forest uses 180 trees and all 170 readings. It does not rebalance the classes.

Split	TN	FP	FN	TP	Error	AUC
Train	59,000	0	0	1,000	0.00%	1.0000
Test	15,608	17	102	273	0.74%	0.9943

Out-of-bag error = **0.58%**; test error = **0.74%**. The test error is 0.16 percentage points higher. The model misses 102 of the 375 failures.

1(d) Random forest with class balancing

A common balancing method is to give rare positive cases more weight. The second forest uses class weights estimated independently for each tree; the remaining forest settings are unchanged.

Split	TN	FP	FN	TP	Error	AUC
Train	59,000	0	0	1,000	0.00%	1.0000
Test	15,608	17	154	221	1.07%	0.9934

Out-of-bag error = **0.77%**; test error = **1.07%**. Compared with the unweighted forest, the weighted version misses 52 more failures and has 17 false alarms. Thus class weighting did not help at the selected decision threshold.

1(e) and 1(f) Corrected model-tree method

The key correction is to use an actual **multivariable split** at each internal tree node. That node fits a logistic regression using all 170 input readings, with an L1 penalty to reduce unnecessary coefficients. Its weighted score decides which branch an observation follows. The terminal group reports an estimated failure probability.

To limit unusually large values, the code applies a signed log transformation followed by standard scaling. A tree can grow to depth 3; a proposed split needs enough rows on both sides. No extra class weights or balancing are used in 1(e).

Library distinction: Standard XGBoost does not create these logistic-regression split rules. Its tree splits use individual features and its L1 setting regularizes leaf outputs. Therefore, the primary results in 1(e) and 1(f) are from the custom L1 logistic model tree that implements the described node rule. Conventional XGBoost is also reported separately for comparison, without confusing the two methods.

Selecting the L1 penalty using five folds

The candidate alpha values are 0.0001, 0.0003, and 0.001. The training data is divided into five balanced folds; each candidate is scored on untouched validation rows. The alpha with the best mean validation AUC is selected. No model setting is chosen using the test set.

Data	alpha	Validation AUC	Validation error	Failure recall
Original	0.0001	0.9194	0.93%	58.40%
Original	0.0003	0.9213	0.91%	61.50%
Original	0.001	0.9186	0.94%	60.00%
SMOTE	0.0001	0.9471	2.73%	86.60%
SMOTE	0.0003	0.9527	3.06%	88.30%
SMOTE	0.001	0.9575	3.26%	90.90%

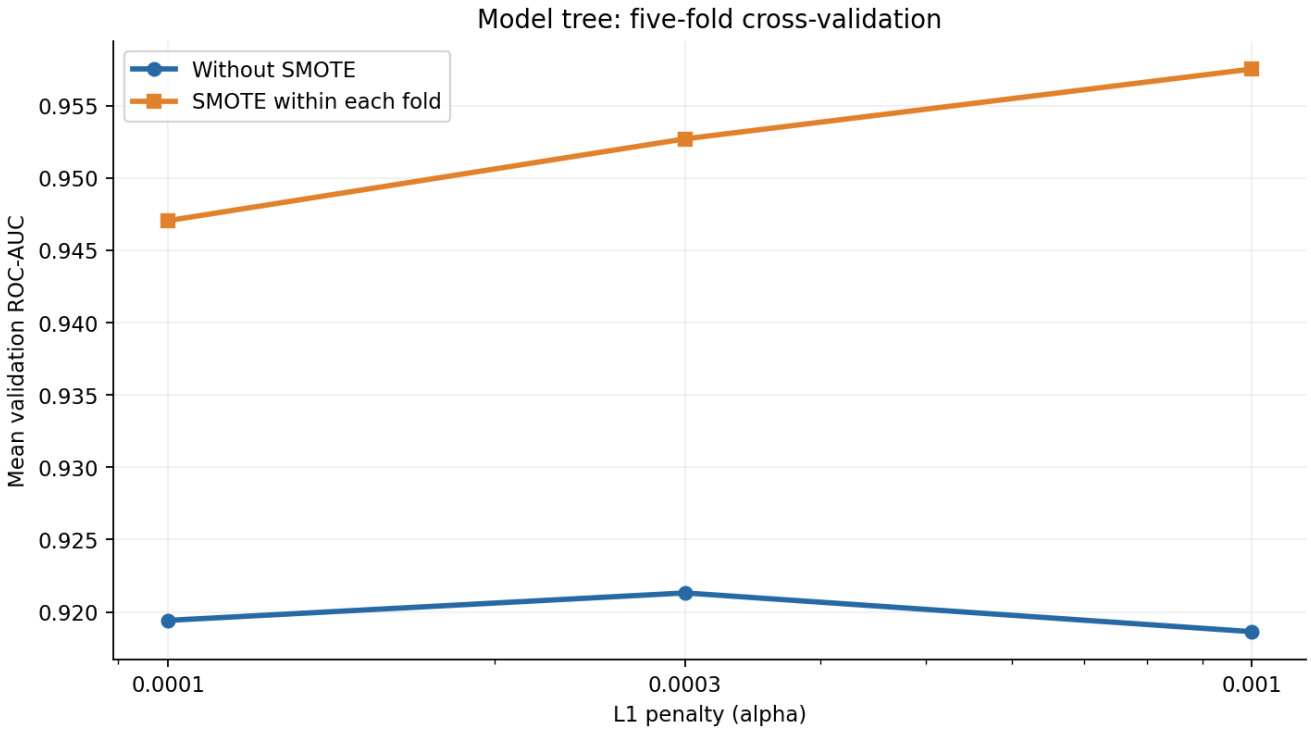


Figure 7. Five-fold model-tree validation AUC across the candidate L1 penalties.

1(e) L1 logistic model tree: original class balance

Cross-validation selects **alpha = 0.0003**. The final tree contains 5 logistic-regression split nodes. No SMOTE or class weights were used.

Split	TN	FP	FN	TP	Error	AUC
Train	58,947	53	418	582	0.78%	0.9343
Test	15,597	28	171	204	1.24%	0.9266

Mean five-fold validation error = 0.91%; final test error = 1.24%. Validation AUC = 0.9213; test AUC = 0.9266. The model correctly detects 204 failures and misses 171.

1(f) L1 logistic model tree: SMOTE inside each fold

SMOTE creates extra positive examples by combining existing positive rows. **Correct order:** first split into folds, then learn imputation and scaling from a fold's training rows, then apply SMOTE to those training rows only, and finally score untouched validation rows. For the last fit, SMOTE is applied to all 60,000 original training rows, producing 59,000 negative and 59,000 positive training rows. The original test set is never changed.

Cross-validation selects **alpha = 0.001**. The final tree contains 7 logistic-regression split nodes.

Split	TN	FP	FN	TP	Error	AUC
Train	57,184	1,816	17	983	3.06%	0.9867
Test	15,166	459	23	352	3.01%	0.9766

Mean validation error = 3.25%; final test error = 3.01%. Validation AUC = 0.9575; test AUC = 0.9766.

Comparison: SMOTE raises caught failures from 204 to 352 and reduces missed failures from 171 to 23. False alarms increase from 28 to 459, so its total error is higher despite detecting more failures. This trade-off matters in real fault detection.

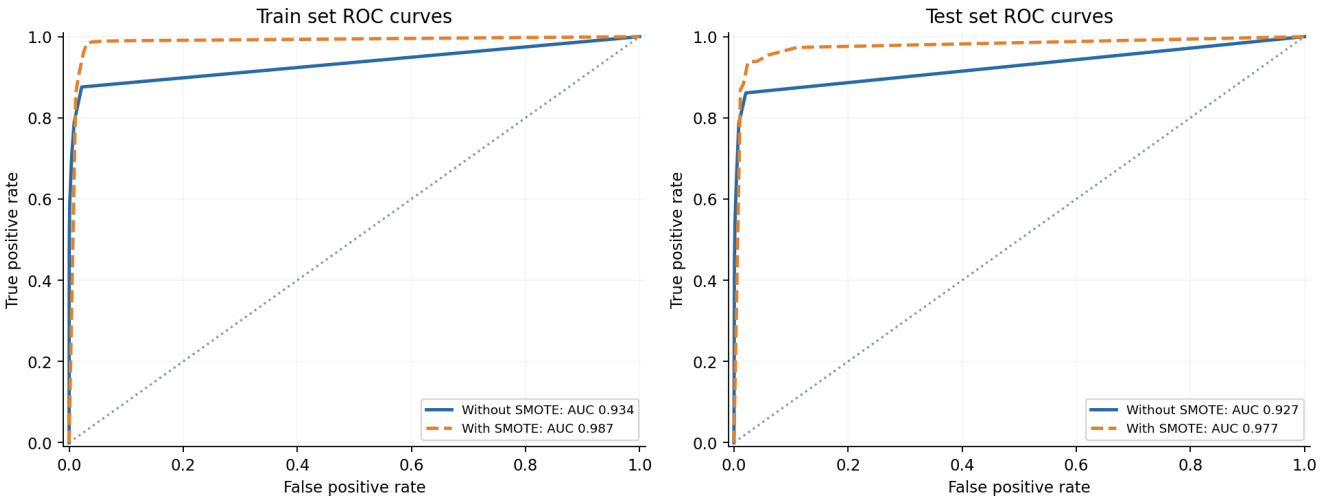


Figure 8. Training and test ROC curves for the corrected L1 logistic model trees.

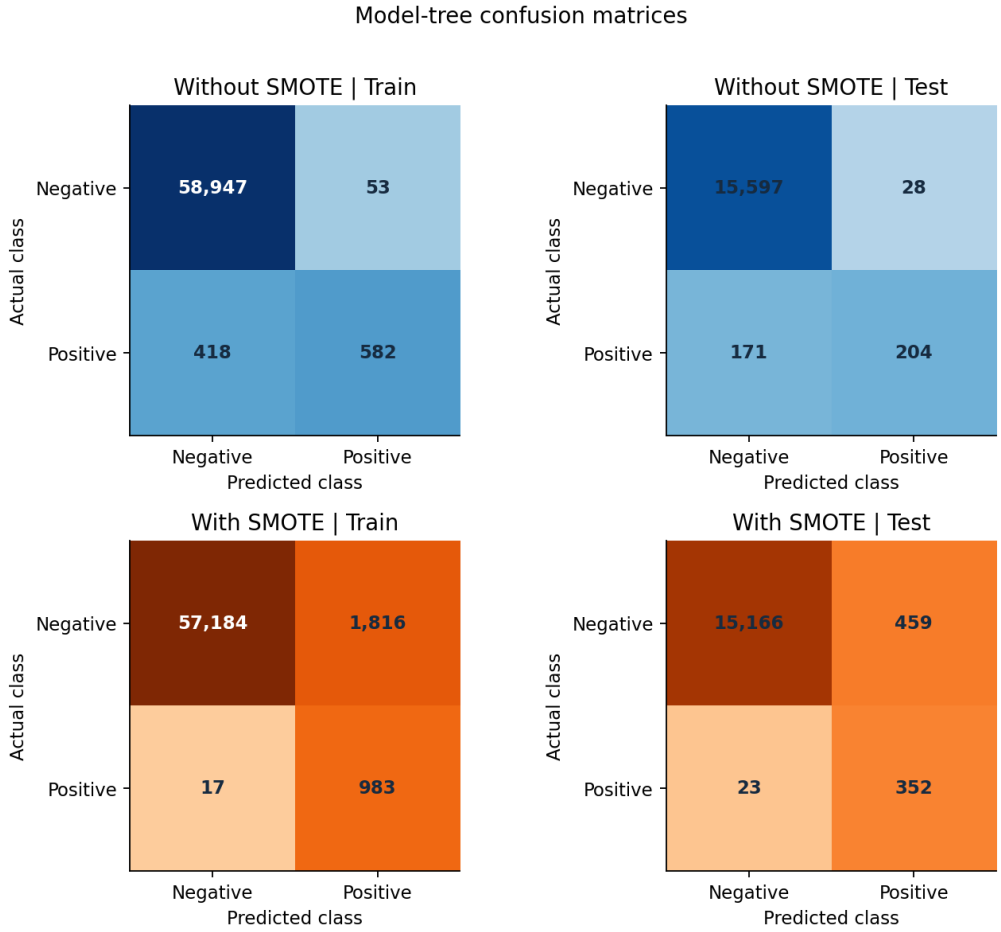


Figure 9. Confusion matrices for both model trees on training and test data.

Additional comparison: conventional XGBoost

The following numbers come from an ordinary XGBoost tree classifier with L1-regularized leaf weights (not logistic-regression split nodes). Five-fold validation chose its L1 settings from 0.1, 1, and 10. This comparison is included because the handout also names XGBoost, but its results are **not** used as a substitute for the requested model-tree calculations.

Model	Train AUC	Test AUC	Test error	Missed	False alarms
Standard XGBoost	0.9986	0.9958	0.70%	98	14
XGBoost + SMOTE	0.9966	0.9942	1.63%	36	225

The conventional XGBoost runs selected alpha = 0.1 without SMOTE and alpha = 10.0 with SMOTE. Their train/test confusion matrices and ROC curves, five-fold validation results, and executable code are retained in the package as supplementary evidence.

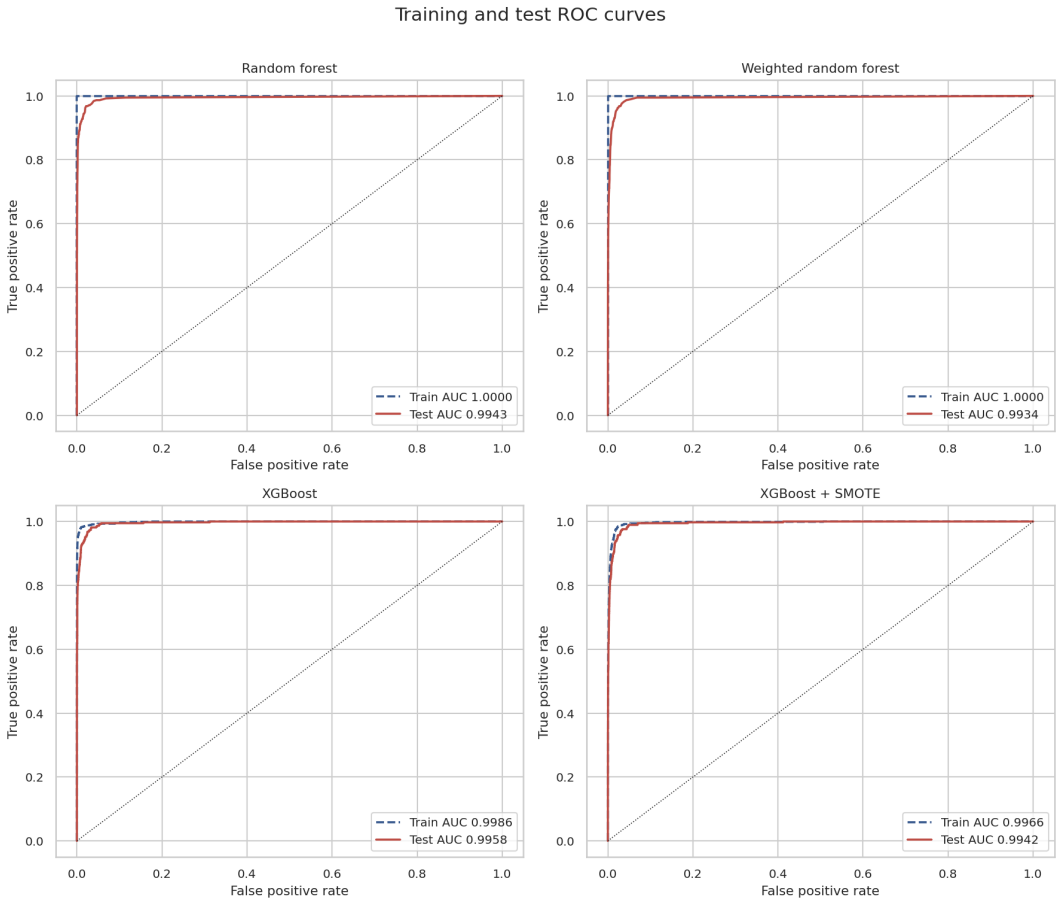


Figure 10. ROC curves for the original forest and conventional XGBoost comparisons.

Summary of the test results

Model	Test error	AUC	Missed	False alarms	Recall
Random forest	0.74%	0.9943	102	17	72.80%
Weighted forest	1.07%	0.9934	154	17	58.93%
L1 logistic model tree	1.24%	0.9266	171	28	54.40%
L1 model tree + SMOTE	3.01%	0.9766	23	459	93.87%
Conventional XGBoost	0.70%	0.9958	98	14	73.87%
XGBoost + SMOTE (extra)	1.63%	0.9942	36	225	90.40%

The custom logistic model tree matches the requested split structure. Conventional XGBoost scores higher on AUC here, but its split structure is different. The results should not be interpreted as identical methods.

For an optional illustration of the APS costs in the source description, false alarms cost 10 units each and missed failures cost 500 units. Under that rule, the two L1 model trees cost 85,780 and 16,090 units, respectively. This cost was **not** used for tuning the models.

2. ISLR 6.6.3 - Lasso constraint

The allowed total absolute size of the coefficients is increased (s gets larger), giving the model more freedom.

Quantity	Choice	Why
(a) Training RSS	iv - Decreases	A model with more freedom can fit training data at least as well, or equally well.
(b) Test RSS	ii - U-shaped	Some flexibility helps at first; too much may fit noise.
(c) Variance	iii - Increases	A flexible fit can change more when the training data changes.
(d) Squared bias	iv - Decreases	The fit can capture more of the true pattern.
(e) Irreducible error	v - Constant	Unpredictable noise is not changed by s.

These are the usual broad patterns intended by the exercise; a single real dataset may behave differently.

3. ISLR 6.6.5 - Identical predictors

Let the two predictors be identical, taking the values a and -a across the two observations, with responses b and -b. Define t = beta1 + beta2. The fitted value depends only on t, not on the separate coefficients.

(a) Ridge objective

Minimize $2(b - a\,t)^2 + \text{lambda}(\text{beta1}^2 + \text{beta2}^2)$.

(b) Ridge coefficients

For any fixed t, the squared penalty is smallest when beta1 = beta2 = t/2. The resulting ridge estimates are beta1 = beta2 = $2ab/(4a^2 + \text{lambda})$, for positive lambda.

(c) Lasso objective

Minimize $2(b - a\,t)^2 + \text{lambda}(|\text{beta1}| + |\text{beta2}|)$.

(d) Why the lasso answers need not be unique

When the two coefficients have the same sign, their absolute values add to |t|. Thus (0,t), (t/2,t/2), and (t,0) all give the same predictions and penalty. The best total is sign(b/a) times $\max(|b/a| - \text{lambda}/(4a^2), 0)$, when a is nonzero. If this total is nonzero, there are infinitely many equally good coefficient pairs. If it is zero with positive lambda, (0,0) is the unique pair.

4. ISLR 8.4.5 - Bagging: votes versus probabilities

Six of the ten trees predict the red class with probability above 0.50, while four predict below 0.50. **Majority voting predicts Red.** The ten probabilities add to 4.50, giving an average of 0.45. **Averaging probabilities predicts Green.** Voting counts the winners only, while averaging uses the confidence of every tree.

5. ISLR 9.7.3 - Maximum-margin separator

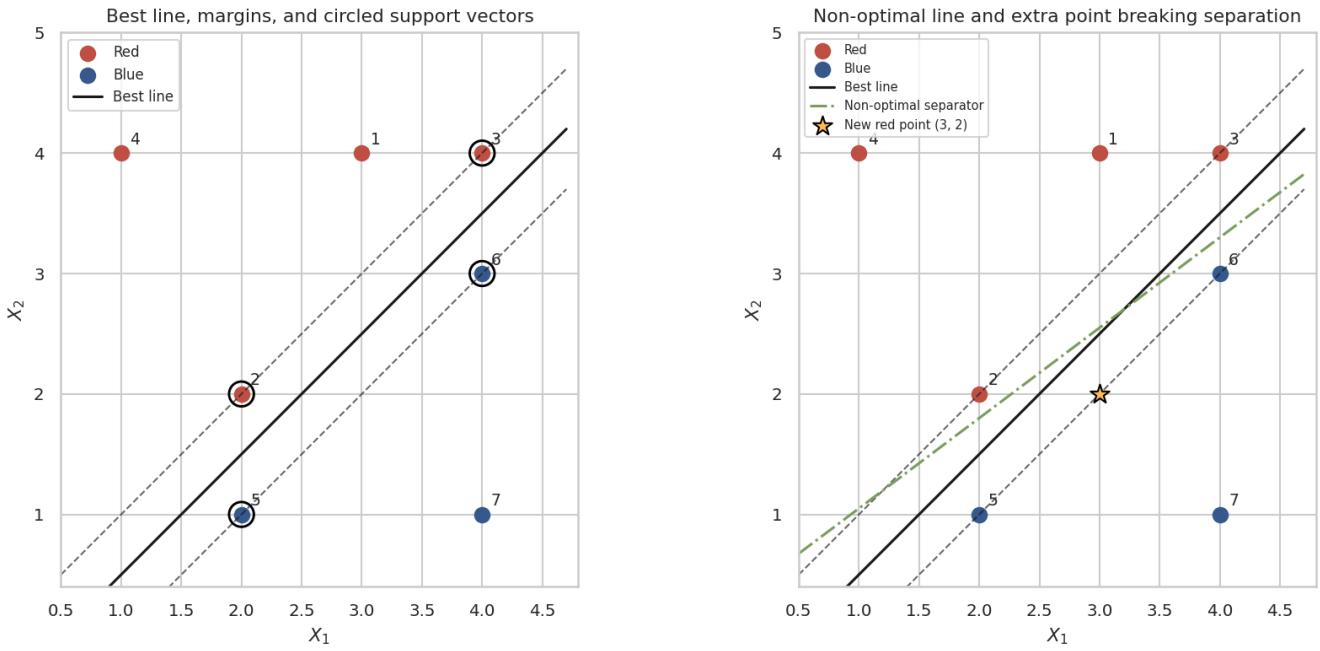


Figure 11. Maximum-margin line (solid), two margins (dashed), alternative line and a new point.

(a) Red points: (3,4), (2,2), (4,4), (1,4). Blue points: (2,1), (4,3), (4,1).

(b) The maximum-margin separating line is X2 = X1 - 0.5, or 0.5 - X1 + X2 = 0.

(c) Predict Red when 0.5 - X1 + X2 is positive, and Blue otherwise. One valid coefficient set is beta0 = 0.5, beta1 = -1, beta2 = 1.

(d) The two margin boundaries are X2 = X1 and X2 = X1 - 1. Each is 0.5/sqrt(2) (about 0.354) from the separating line.

(e) Support vectors: Red points 2 and 3; Blue points 5 and 6. They lie on the two margin boundaries.

(f) Moving point 7 slightly will not change the maximum-margin line, because it does not touch either margin.

(g) The line X2 = 0.75 X1 + 0.30 also separates the original groups but has a smaller margin, so it is not optimal.

(h) Add a Red point at (3,2). This falls on the segment between two Blue points, so no straight line can then separate all Red points from all Blue points.

6. Extra practice (optional)

ISLR 5.4.2 - Bootstrap inclusion chance

(a)-(c) Any one draw misses observation j with probability (n-1)/n. All n independent draws miss it with probability $(1-1/n)^n$; therefore its probability of appearing at least once is $1 - (1-1/n)^n$.

(d) n = 5: 0.67232. (e) n = 100: 0.63397. (f) n = 10,000: 0.63214. (g) The value approaches 1-1/e = 0.63212 as n grows. (h) A simulation of 10,000 bootstrap samples of size 100 produced a proportion of 0.6304, close to the theoretical 0.6340.

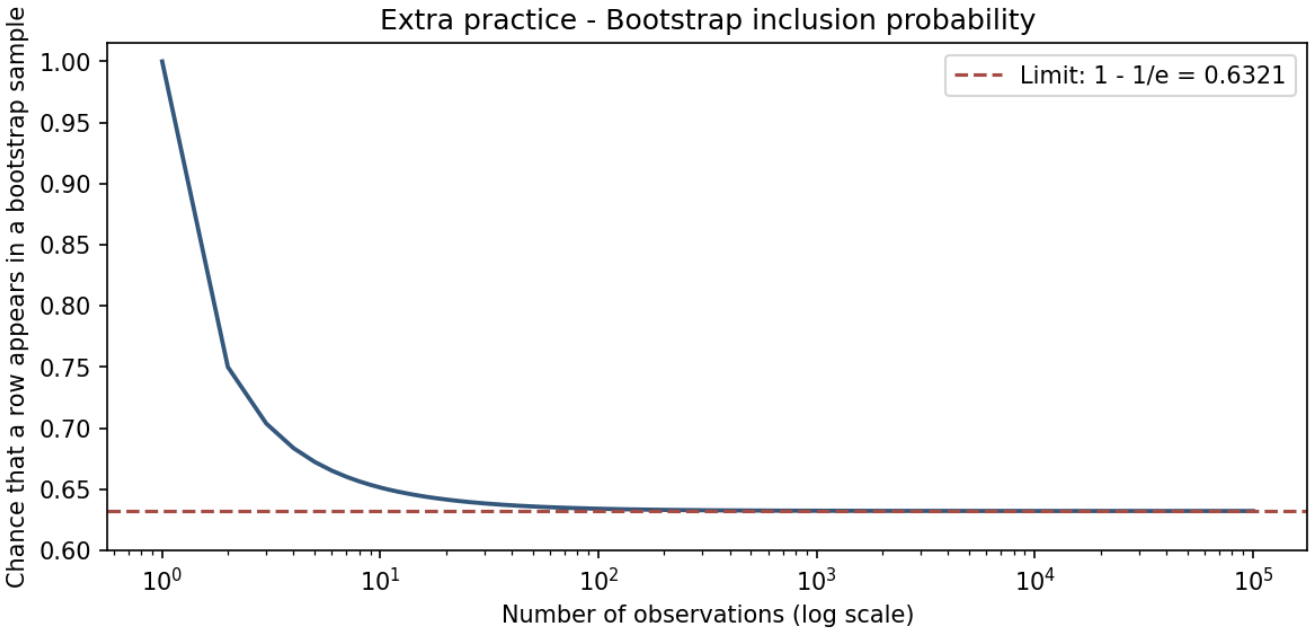


Figure 12. Probability that an observation appears at least once in a bootstrap sample.

ISLR 6.8.4 (first-edition numbering) - Ridge as lambda increases

(a) Training RSS: iii, increases. (b) Test RSS: ii, usually U-shaped. (c) Variance: iv, decreases. (d) Squared bias: iii, increases. (e) Irreducible error: v, remains constant.

ISLR 8.4.4 - Tree partitions

(a) The pictured split puts X1 at least 1 in a region with prediction 5. On the left, a cut at X1 = 0 and then X2 = 1 gives the four remaining terminal values 15, 3, 0, and 10. (b) For the provided tree, first split at X2 = 1. For X2 below 1, split on X1 = 1 into values -1.80 and 0.63. For X2 between 1 and 2, split on X1 = 0 into -1.06 and 0.21. At X2 at least 2, predict 2.49.

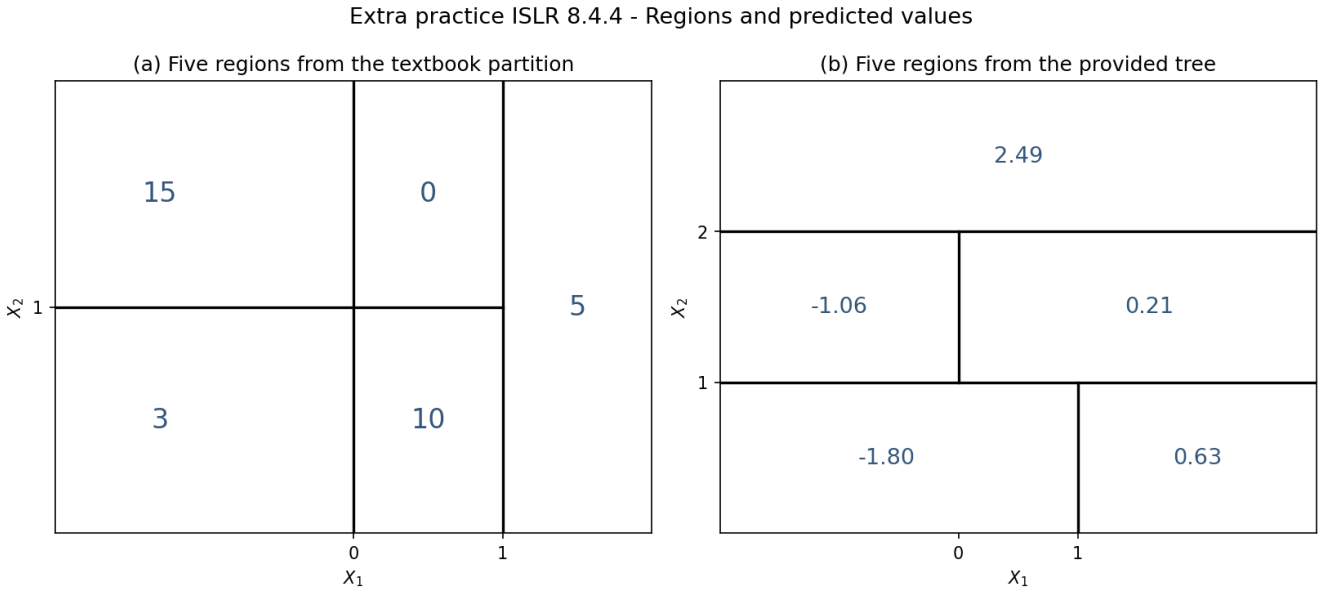


Figure 13. Regions corresponding to the two decision-tree questions.

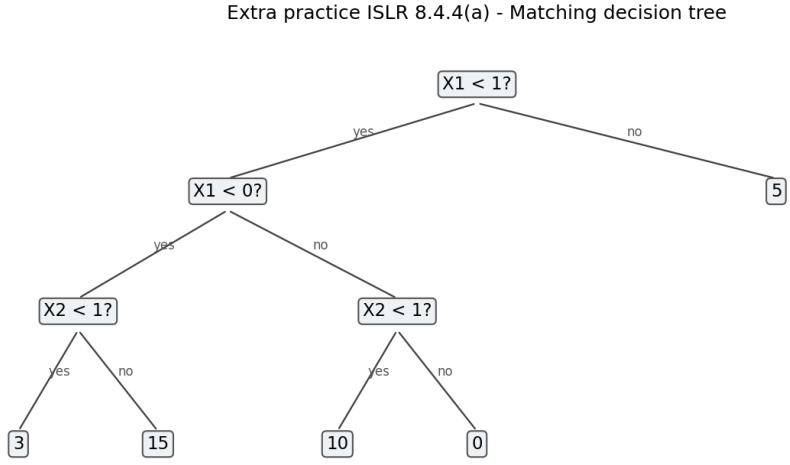


Figure 14. Tree matching the first partition diagram.

ISLR 9.7.2 - Nonlinear decision boundary

(a) $(1 + X1)^2 + (2 - X2)^2 = 4$ is a circle with center (-1,2) and radius 2. (b) Values above 4 are outside; values at most 4 are inside or on the circle. (c) (0,0) Blue; (-1,1) Red; (2,2) Blue; (3,8) Blue. (d) Expanding gives $1 + 2\ X1 + X1^2 - 4\ X2 + X2^2 = 0$, a linear rule in the expanded inputs X1, $X1^2$, X2 and $X2^2$.

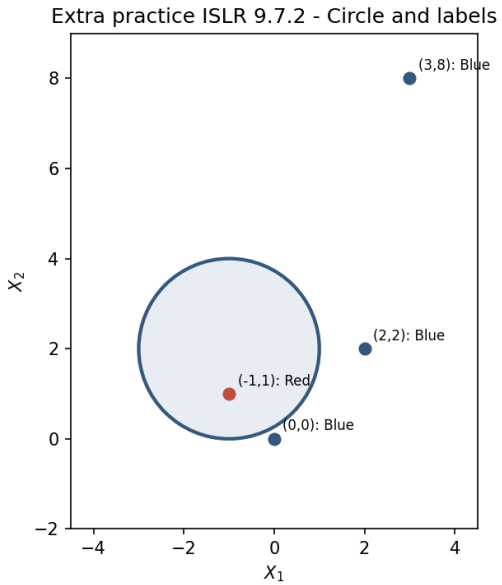


Figure 15. The circular boundary and four example observations.

References

[1] DSCI 552 Homework 6, user-supplied question sheet, and the accompanying APS training and test files.

[2] UCI Machine Learning Repository: APS Failure at Scania Trucks.
https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks

[3] James, Witten, Hastie and Tibshirani. An Introduction to Statistical Learning, first and second editions.
https://www.statlearning.com/

[4] XGBoost documentation, Parameters and Introduction to Boosted Trees:
https://xgboost.readthedocs.io/en/stable/parameter.html and
https://xgboost.readthedocs.io/en/stable/tutorials/model.html

[5] scikit-learn documentation, SGDClassifier:
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

AI use acknowledgment

AI tools were used to assist with organizing the code, compiling the report, and checking formatting. All reported numerical results were generated from the included APS datasets using the accompanying reproducible code.